Examen técnico NestJS / Node.js

Objetivo

Desarrollar una API REST en **NestJS** o **Node.js + Express** para registrar y administrar empleados.

La aplicación deberá utilizar obligatoriamente:

- TypeScript.
- Arquitectura por módulos: controller, service, module, dto y entities.
- Autenticación con JWT.
- Middlewares propios.
- Datos mock locales (un arreglo en memoria dentro del service).

No es necesario utilizar una base de datos real.

Estructura del proyecto

El proyecto deberá organizarse por módulos. Cada módulo agrupa su controller, service, module, DTOs y entities.

Responsabilidad de cada archivo:

- **controller:** recibe las peticiones y llama al service. No debe contener lógica.
- **service:** contiene la lógica y maneja los datos mock.
- **module:** agrupa el controller y el service del módulo.
- **dto:** define y valida los datos de entrada (por ejemplo, con class-validator).
- **entities:** define el modelo del empleado.

Empleado

Cada empleado deberá tener:

- `id` (generado por el servidor)
- `nombreCompleto`
- `area`: TECNOLOGIA, RECURSOS_HUMANOS, FINANZAS, OPERACIONES, VENTAS o ADMINISTRACION
- `puesto`
- `genero`: MASCULINO, FEMENINO u OTRO
- `fechaEntrada` (YYYY-MM-DD)

- `fechaNacimiento` (YYYY-MM-DD)
- `activo` (true / false; por defecto true)

Las respuestas deberán incluir también el campo `estado` con el valor "Activo" o "Inactivo".

El proyecto deberá iniciar con al menos 5 empleados mock.

Autenticación

Crear el endpoint `POST /api/auth/login` que reciba usuario y contraseña y devuelva un token JWT.

Usuarios mock:

- `admin` / `Admin123!` → rol **ADMIN** (puede consultar, registrar y activar/desactivar).
- `consulta` / `Consulta123!` → rol **CONSULTA** (solo puede consultar).

Reglas:

- Todas las rutas de empleados requieren el encabezado `Authorization: Bearer <token>`.
- Sin token, token inválido o expirado → **401**.
- Rol sin permiso → **403**.
- La clave del JWT deberá leerse desde un archivo `.env`.

Middlewares

Se deberán implementar al menos:

- **Logger:** mostrar en consola el método, la ruta, el código de respuesta y el tiempo en ms.
- **Autenticación:** validar el token JWT.
- **Roles:** validar que el usuario tenga permiso para la operación.
- **Manejo de errores:** responder todos los errores con el mismo formato.

Endpoints

Método	Ruta	Acceso	Descripción
`POST`	`/api/auth/login`	Público	Inicia sesión.
`GET`	`/api/empleados`	ADMIN, CONSULTA	Lista los empleados.
`POST`	`/api/empleados`	ADMIN	Registra un empleado.
`PATCH`	`/api/empleados/:id/estado`	ADMIN	Activa o desactiva un empleado.

Registrar empleado

Al recibir la petición:

1. Validar que todos los campos sean obligatorios y tengan valores válidos. Si no, responder **400**.
2. Crear el empleado como activo.
3. Guardarlo en los datos mock.
4. Responder **201** con el empleado creado.

Al consultar `GET /api/empleados`, el nuevo empleado deberá aparecer en la lista.

Activar / desactivar empleado

- Debe actualizar `activo` y `estado` ("Activo" / "Inactivo").
- Si el empleado no existe → **404**.

Entregable

Se deberá entregar:

1. Proyecto completo con su código fuente.
2. Archivo `.env.example`.
3. Archivo `README.md` con:
    - Instrucciones para ejecutar el proyecto.
    - Versión de Node.js y del framework utilizado.
    - Breve explicación de la estructura del proyecto.
    - Dependencias principales.
    - Ejemplos de peticiones (curl o colección de Postman).
4. El proyecto deberá poder ejecutarse con:

```
npm install
npm run start:dev
```